

Outdoor Vision CV App

Image Identity, Fingerprinting, and Standardized Intake Naming

VERSION	DATE	STATUS
1.0	September 3, 2026	Pending Approval
OWNER	APPROVER	PLATFORM
Dave Vercher	Dave Vercher	Windows Desktop

DOCUMENT
INTENT

Implementation-ready requirements for assigning every Intake-imported JPEG a durable UUIDv4 identity, computing an encoded-byte SHA-256 fingerprint during copy, renaming imports into a project-lifetime sequence, and reconciling tracked state through Home-page Refresh.

This document is the authoritative requirements baseline for the Image Identity and Fingerprinting enhancement.

Contents

1. Document Purpose	3
2. Product Vision and Scope	3
3. Governing Concepts and Terminology	3
4. Authoritative Identity Rules	3
5. Standardized Intake Filenames	4
6. Encoded SHA-256 Fingerprinting	4
7. Intake Import Workflow	5
8. Private JSON State Model	5
9. Tracking Lifecycle Inside Intake	6
10. Home-page Refresh and Reconciliation	6
11. Review Master Dataset and Queue Metrics	7
12. Image Details Dialog	7
13. Recovery and Rebuild	7
14. Performance, Error, and Accessibility Requirements	7
15. Required State Transitions	8
16. Suggested Architecture Boundaries	8
17. Acceptance Criteria	8
18. Required Tests	9
19. Risks, Assumptions, and Future Enhancements	9
20. Reference Baselines	9

1. Document Purpose

This document defines the product, persistence, filesystem, user-interface, recovery, performance, and quality requirements for durable JPEG identity inside the Outdoor Vision CV App. It is intended to provide sufficient context for a future implementation session and must be read alongside the current repository and the approved Intake and Home-page PRDs.

Where this document conflicts with the prior Intake or Home-page requirements, this document supersedes only the affected requirements for imported filenames, Intake manifest contents, tracked-image statistics, Refresh reconciliation, and image details. All unrelated approved behavior remains unchanged.

RELEASE
BOUNDARY

This release creates identity and fingerprint infrastructure. It does not detect, report, block, or remove duplicate images.

2. Product Vision and Scope

Intake is the sole supported gateway for bringing images into a project. Each successfully imported JPEG becomes a managed image instance with its own immutable UUID, encoded-byte fingerprint, standardized filename, provenance, and current project-relative location. Matching fingerprints are allowed because identity belongs to the physical imported instance, not to its contents.

2.1 In Scope

- UUIDv4 assignment for each successfully completed Intake copy.
- SHA-256 over the exact encoded JPEG bytes while copying in 1 MiB chunks.
- Standardized project-lifetime filenames from IMG0000001.jpg through IMG9999999.jpg.
- A private, per-project JSON state model that unifies image tracking and Intake queue state.
- Identity-preserving sort, Undo, class rename, class deletion, and same-project Refresh reconciliation.
- Tracked-only project statistics plus separate unregistered-JPEG metrics and locations.
- A read-only Image Details dialog for the current image on the sorting screen.
- Progress, cancellation, backup, interrupted-operation recovery, and manifest rebuild behavior.
- Automated tests, manual GUI verification, and README/internal integration documentation.

2.2 Explicitly Out of Scope

- Duplicate detection, duplicate removal, import-time duplicate checks, or duplicate warnings.
- Perceptual, decoded-pixel, or similarity hashing.
- Automatic hash refresh for tracked files still present at their expected paths.
- In-app movement of images between projects or cross-project identity reconciliation.
- Registration of JPEGs added manually, including files copied or moved into a project outside Intake.
- Tracking-metadata export, SQLite, legacy-data migration, or existing-image backfill.
- PNG, BMP, TIFF, HEIC, RAW, video, or other non-JPEG formats.
- Multi-user synchronization, cloud services, or app-wide duplicate analysis.

3. Governing Concepts and Terminology

Managed image	A JPEG copy successfully imported through Intake and committed to private tracking state.
Image UUID	A lowercase, canonical, hyphenated UUIDv4 identifying one imported physical image instance.
Fingerprint	A lowercase 64-character SHA-256 hexadecimal digest of the file's exact encoded bytes.
Assigned filename	The original Intake sequence name, retained in metadata even if the physical file is later renamed.
Current relative path	The tracked image's present path relative to its registered project root.
Unregistered JPEG	A valid JPEG found inside a project that has no active managed-image record.
Reconciliation	A Refresh process that confirms tracked paths, reconnects unambiguous same-project moves, removes confirmed missing records, and reports unknown files.
Import batch	The valid JPEGs accepted from one confirmed source-folder import.

IDENTITY RULE

Every import or copy through Intake receives a new UUID, even when its SHA-256 matches another image. Moving, sorting, renaming, or undoing a managed image within its project preserves that UUID.

4. Authoritative Identity Rules

ID	Requirement	Priority
ID-01	Generate a cryptographically random UUIDv4 only after an Intake copy has completed successfully and is ready to be committed as a managed image.	Must
ID-02	Store UUID values in lowercase canonical hyphenated form. Reject missing, malformed, or duplicate UUIDs when loading private state.	Must
ID-03	Treat the UUID and SHA-256 fingerprint as independent fields. Equal fingerprints never merge records or reuse UUIDs.	Must
ID-04	Preserve the UUID when the app sorts, renames, moves within the project, or undoes movement of the same managed image.	Must
ID-05	Generate a new UUID for every new Intake-created physical copy, including a copy imported from another registered project.	Must
ID-06	Do not embed the UUID, fingerprint, project name, source filename, or date in the physical filename.	Must
ID-07	Bind each record to the project's stable internal project ID, never solely to its mutable name or absolute path.	Must
ID-08	Treat Intake as the only normal registration boundary. Refresh and ordinary folder scans must not silently adopt unknown files.	Must

5. Standardized Intake Filenames

Every successfully imported copy must use this exact case-sensitive format:

IMG0000001.jpg

ID	Requirement	Priority
NAM-01	Use the literal uppercase prefix IMG followed by exactly seven zero-padded decimal digits and the lowercase extension .jpg.	Must
NAM-02	Allocate sequence numbers per project for the lifetime of that project. Valid values are 0000001 through 9999999.	Must
NAM-03	Assign lower available sequence numbers to earlier items in the confirmed import order. The resulting names must sort lexically and numerically in the same order.	Must
NAM-04	Never intentionally reuse an allocated or uncertain sequence after deletion, cancellation, failure, or recycling. Harmless gaps are allowed.	Must
NAM-05	Before allocation and again before finalization, check the complete project tree case-insensitively for an existing filesystem entry using the candidate name. Skip occupied numbers without altering the existing entry.	Must
NAM-06	If fewer sequence values remain than valid images in the confirmed batch, block the import before copying and explain that the project filename namespace is exhausted.	Must
NAM-07	Normalize both .jpg and .jpeg source extensions to .jpg without decoding, re-encoding, or otherwise changing the JPEG bytes.	Must
NAM-08	Rename only the project-managed copy. Never rename, modify, or delete the external source image.	Must
NAM-09	Show aggregate renamed-to-standard-format counts in Preview and Results. Do not require per-file original-to-generated mappings in the UI.	Must

5.1 Collision During Later Movement

If sorting or Undo encounters an occupied destination name, allocate the next available project sequence, move the managed image under that new standard name, preserve its UUID, fingerprint, and original assigned_filename value, update current_relative_path, and disclose the rename. Never overwrite an existing entry.

6. Encoded SHA-256 Fingerprinting

ID	Requirement	Priority
HSB-01	Compute SHA-256 from the JPEG's exact encoded byte stream, not from decoded pixels, EXIF-normalized pixels, thumbnails, filenames, or metadata fields.	Must
HSB-02	Read and hash the source in 1 MiB chunks, where READ_SIZE equals 1024 x 1024 bytes, while writing the project copy.	Must
HSB-03	Use one continuously updated SHA-256 digest for the complete file. Store the final lowercase 64-character hexadecimal digest.	Must
HSB-04	The destination copy must remain byte-for-byte identical to the source. Preserve EXIF and other encoded content; filename extension normalization must not re-encode the file.	Must
HSB-05	Do not compare the newly generated digest with existing digests during import and do not warn when fingerprints match.	Must
HSB-06	If source reading or hashing fails for one candidate, apply the approved Intake per-file failure behavior. A partial temporary destination must not become a managed image.	Must
HSB-07	Do not recompute a tracked image's hash merely because Home-page Refresh found it at its recorded path.	Must

**VERIFICATION
RULE**

For a given source file, the stored fingerprint must equal the standard SHA-256 digest of its complete encoded byte sequence. Matching fingerprints mean the compared encoded files are byte-for-byte identical for this product's duplicate definition.

7. Intake Import Workflow

7.1 Source Boundary

- Continue accepting one external source folder through Intake's established browse or folder-drop flow.
- Reject a source anywhere inside the destination project to prevent self-import and uncertain identity.
- Allow a folder inside Project A to be selected as the source for Project B. This is a copy: Project A remains unchanged and every Project B copy receives a new UUID and sequence.
- Continue validating candidate contents as JPEG before confirmation. Do not trust extensions alone.

7.2 Preview

- Keep Preview aggregate-only. Show valid JPEGs, invalid/unreadable candidates, hidden/system skips, estimated bytes, destination, available sequence capacity, and files that will be renamed to the standard format.
- Do not show UUIDs, hashes, or per-file original-to-generated name mappings in Preview.
- No UUID generation, hashing, copying, or sequence commitment may begin until the user confirms Import.

7.3 Copy, Hash, and Commit

1. Revalidate the source, destination, sequence capacity, and candidate filename immediately before copying.
2. Reserve the next available sequence under the per-project operation lock. Persist enough pending-operation state for crash recovery; gaps are acceptable.
3. Stream the source once in 1 MiB chunks, updating SHA-256 while writing to a temporary file in the destination filesystem.
4. Flush and close the temporary file, preserve modification time/basic metadata where Windows permits, and confirm expected byte count.
5. Finalize the standard filename without overwriting any entry.
6. Create the UUIDv4 and commit the managed-image record and queue state atomically in the private manifest.
7. Report the image as copied only after the file and durable record agree. Remove or recover any incomplete temporary artifact.

7.4 Cancellation and Partial Results

Completed images remain imported with their UUIDs, hashes, sequence names, and queue records. The active incomplete temporary file is removed. Later candidates are not started. Reserved or uncertain sequence numbers may remain unused. Results must accurately report completed, failed, cancelled, invalid, renamed-to-standard-format, bytes copied, and destination counts without duplicate-check language.

8. Private JSON State Model

Use private per-user application data keyed by stable project ID. JSON is the required persistence format for this enhancement; SQLite is explicitly out of scope. The storage implementation must remain behind a service boundary so UI code does not directly manipulate records.

8.1 Unified Per-Project Manifest

Evolve the existing Intake manifest into the single authoritative per-project state for all managed images and active queue behavior. Class mappings may remain in mappings.json because mapping changes do not alter image identity. The exact property names may follow repository conventions, but the following information is required.

Manifest	Schema version, project ID, next filename sequence, image collection, import-batch collection, queue state/order, pending recovery operation, and last successful reconciliation time.
Managed image	UUID, SHA-256, assigned standard filename, current relative path, lifecycle state, original source filename, original source-relative path, batch ID, import timestamp, encoded byte size, and dimensions when available.
Import batch	Batch UUID/ID, UTC timestamp, historical absolute source root, candidate/result counts, and aggregate bytes.
Queue	Only unsorted managed images participate. Preserve current approved batch ordering, skip behavior, and resumability.
Recovery	Operation type, affected UUID/path(s), pre-operation values, intended values, and phase sufficient to complete or roll back after interruption.

8.2 Canonical Storage Rules

- Store timestamps in UTC ISO 8601 form and render them in the user's local Windows locale/time zone.
- Store project-relative paths in one documented normalized form and compare Windows paths case-insensitively.
- Validate schema version, project ID binding, UUID uniqueness, hash syntax, relative-path safety, sequence range, and queue references on load.
- Reject absolute or parent-traversing current-relative paths. A corrupt or untrusted manifest blocks changing operations until recovery.
- Use per-project reentrant locking and atomic temporary-file, flush, fsync, and replace behavior for every save.

8.3 Last-Known-Good Backup

Maintain one private last-known-good backup. Validate the active manifest before promoting it to the backup so a corrupt active file cannot replace the recoverable copy. Recovery must try the active file, then the backup, before offering a destructive rebuild.

9. Tracking Lifecycle Inside Intake

ID	Requirement	Priority
LIF-01	Sorting must move the physical JPEG and update its current relative path in the same logical operation while preserving UUID, SHA-256, provenance, and assigned sequence.	Must
LIF-02	Undo must restore the image to intake/unsorted, restore queue state, and update the current path while preserving identity.	Must
LIF-03	If Undo or sorting requires a collision-driven standard rename, allocate a new unused sequence, persist the advanced counter and current path, retain the UUID/hash/original assigned filename, and notify the user.	Must
LIF-04	Renaming a class must update every managed current-relative path beneath that direct class only after the folder rename succeeds.	Must
LIF-05	After successful recycling of a class, permanently remove managed-image records for all registered files that were inside it. If recycling fails, retain the records and mapping.	Must
LIF-06	Skip changes queue order only and must not change UUID, hash, filename, path, or provenance.	Must
LIF-07	Sorted-image records remain in the manifest indefinitely unless their images are deleted, confirmed missing by Refresh, or lost during an explicitly disclosed rebuild.	Must
LIF-08	No operation may report success while its physical file and durable tracking state disagree.	Must

9.1 Persistence Failure

For reversible operations, attempt to restore the pre-operation filesystem and manifest state if persistence fails. If rollback also fails, preserve a recovery record, block conflicting Intake actions, and require recovery. For class deletion, persist intent before recycling; after successful recycling, complete tracking cleanup automatically or during next-launch recovery.

10. Home-page Refresh and Reconciliation

Refresh is the explicit audit boundary for managed-image paths and project metrics. It must run outside the GUI thread, expose a progress bar and Cancel button, and preserve pre-refresh metadata if cancelled or if the project scan is incomplete.

10.1 Scan Rules

1. Scan each registered project root recursively without following directory symlinks, junctions, reparse points, or paths outside the project.
2. Check every managed image at its recorded path. Existing files remain confirmed without hash recomputation.
3. Collect otherwise-unregistered .jpg/.jpeg candidates and validate their contents. Report invalid/unreadable candidates separately and never hash or register them.
4. For each missing record, narrow unregistered candidates by encoded size and compute candidate SHA-256 values in 1 MiB chunks only as needed.
5. When exactly one missing record matches exactly one unregistered candidate inside the same project, update only the current relative path and preserve UUID, hash, provenance, and assigned sequence. Retain the candidate's actual filename; Refresh does not rename files.
6. When matching is ambiguous, retain the missing record, register no candidate, and report every affected record and candidate path in expandable details.
7. When a missing record has no candidate anywhere in the same project, remove it automatically after the full project scan succeeds.
8. Never search other projects for a missing image, transfer identity across projects, or register an unknown file.

10.2 All-or-Nothing Application

Stage reconciliation results in memory. If Refresh is cancelled or a project cannot be scanned completely because of access, I/O, validation, or persistence failures, apply no path updates or missing-record removals for that project. Other independently completed projects may commit successfully and must be identified accurately in the final summary.

10.3 Metrics and Results

Managed total	Count only active managed-image records whose files are confirmed present. This replaces recursive extension-only JPEG totals as the primary project image count.
Labeling denominator	Use the managed-image total. Clamp inconsistent labeled values for display as established by the Home-page PRD.
Unregistered count	Count valid JPEGs with no managed record and persist the most recent successful count per project.
Project card	Show the managed total and a compact persistent warning such as '3 unregistered JPEGs' when nonzero.
Refresh summary	Show tracked confirmed, moves/renames reconciled, missing records removed, unregistered valid JPEGs, invalid/unreadable candidates, ambiguous matches, and failed projects/files.
Expandable details	List locations for unregistered JPEGs, invalid/unreadable candidates, ambiguous records/candidates, removals, and errors.

11. Review Master Dataset and Queue Metrics

ID	Requirement	Priority
MET-01	The Intake landing page's unsorted count must include only present, managed images in the active queue.	Must
MET-02	Review Master Dataset must count only present, managed JPEGs assigned to each direct class folder.	Must
MET-03	Review must show a separate warning/count when unregistered JPEGs exist in reviewed class locations and must not include them in managed totals.	Must
MET-04	Files with JPEG extensions that fail content validation are not managed or unregistered JPEGs; report them as invalid/unreadable scan findings.	Must
MET-05	Project statistics must update after successful import, classification, Undo, class deletion, recovery, or completed Refresh as appropriate.	Must

12. Image Details Dialog

Add an Image Details action to the existing sorting-screen toolbar near Open Image Externally. It opens a read-only dialog for the current unsorted image only. Browsing details for sorted images is out of scope.

ID	Requirement	Priority
UI-01	Display the standard/current filename, full UUID, full SHA-256, current project-relative path, original source filename, original source-relative path, import batch and timestamp, encoded size, and image dimensions.	Must
UI-02	Label the digest exactly as 'SHA-256 (encoded file bytes)' so it cannot be confused with a perceptual or decoded-pixel hash.	Must
UI-03	Keep values read-only but text-selectable. Provide dedicated Copy UUID and Copy SHA-256 buttons.	Must
UI-04	Use wrapping or horizontally safe presentation so the UUID, 64-character digest, and long paths remain readable without clipping at minimum supported size and common Windows scaling.	Must
UI-05	Opening or copying details must not move, skip, classify, rename, rehash, or otherwise modify the image.	Must
UI-06	Keyboard focus inside the dialog must suppress sorting hotkeys. Escape closes safely; copy failure is reported without closing the dialog.	Must
UI-07	If the current file cannot render, show all available stored metadata and the established render error independently.	Must

13. Recovery and Rebuild

13.1 Interrupted Operations

On startup or Intake entry, detect pending import, sort, Undo, class rename, or deletion operations. Attempt deterministic completion or rollback before enabling conflicting controls. Recovery is automatic and quiet when the pre-operation state is restored exactly; otherwise show a concise result or actionable error. Never create a second UUID for the same pending import.

13.2 Manifest Recovery Order

1. Validate and load the active manifest.
2. If active state is corrupt or unreadable, validate and restore the last-known-good backup.
3. If both fail, block importing, sorting, Refresh changes, class rename, and class deletion and offer Rebuild Tracking Records.

13.3 Rebuild Tracking Records

- Require explicit confirmation that prior UUIDs, original paths, and batch history cannot be recovered.
- Scan only direct JPEG files in intake/unsorted and direct class folders under intake/sorted; ignore nested class folders and report them according to existing rules.
- Validate each JPEG, calculate a new current SHA-256, and generate a new UUIDv4.
- Preserve all physical filenames. Infer assigned sequences only from valid IMG#####.jpg names and set the next counter above the highest recognized sequence.
- Register valid direct unsorted images as one recovered batch ordered case-insensitively by filename. Register valid direct sorted images under their containing class.
- Set unavailable provenance fields to explicit recovered/unknown values rather than fabricating source information.
- Write rebuilt state and backup atomically, then present registered, omitted, invalid, and warning counts.

14. Performance, Error, and Accessibility Requirements

- Design and verify normal operation for approximately 5,000 to 10,000 managed images per project.
- Run import, hashing, recursive Refresh, recovery hashing, class deletion, and bulk class-path updates outside the Tkinter GUI thread.

- Update import and Refresh progress at least once per second when measurable. Include processed/total files and bytes when known.
- Refresh must provide a visible progress bar and Cancel button. Disable or debounce repeated Refresh actions while one is active.
- Use cancellation checks between chunks/files and at safe commit boundaries; never leave a partial file presented as a managed image.
- Disable conflicting filesystem-changing controls while a protected operation or recovery is active.
- Present errors in plain language with the affected project/path and a useful next action. Do not expose raw JSON as the only explanation.
- All new controls and dialogs must be keyboard accessible, have clear focus/disabled states, and remain usable at the documented minimum size and common Windows scaling.
- Do not follow unsafe filesystem links or permit relative-path traversal from private metadata.

15. Required State Transitions

intake_preview	Discovery complete; aggregate standard-naming and capacity information shown; no identity allocated.
intake_copyping	Files stream to temporary destinations while hashes and progress are calculated.
intake_committing	Final path and durable managed-image record are being synchronized.
intake_recovery_required	Private state or a prior operation cannot be trusted; conflicting actions are blocked.
refresh_scanning	Registered projects and candidate files are scanned in the background.
refresh_reconciling	Complete scan results are being validated and atomically applied per project.
refresh_cancelled	No incomplete project's staged changes are applied; prior durable state remains authoritative.
image_details	Read-only metadata dialog is active and sorting hotkeys are suppressed.

16. Suggested Architecture Boundaries

- Image tracking service: identity creation, record validation, path updates, removal, lookup, and UI-safe read models.
- Intake state store: schema-versioned JSON load/save, per-project locks, last-known-good backup, atomic replacement, and recovery records.
- Filename allocator: project-lifetime sequence reservation, case-insensitive collision scan, exhaustion validation, and non-reuse behavior.
- Streaming copy/fingerprint service: temporary destination, 1 MiB read/write/hash loop, metadata preservation, cancellation, and cleanup.
- Reconciliation service: safe recursive scan, candidate validation, size filtering, on-demand hashing, ambiguity detection, staged results, and per-project commit.
- Statistics integration: managed/unregistered counts, labeling denominator, project-card warning, and completion summaries.
- Intake UI/controller: aggregate Preview/Results, progress/cancel, Image Details dialog, and recovery messages only; no direct JSON mutation.

IMPLEMENTATION CONSTRAINT

The exact class and method names may follow repository conventions, but filesystem services and state transitions must remain testable without launching the GUI.

17. Acceptance Criteria

1. Every successfully completed Intake copy receives a unique UUIDv4 and a SHA-256 matching the exact encoded source bytes.
2. Two byte-identical imports receive different UUIDs and matching hashes without a duplicate warning.
3. Imported copies are named IMG0000001.jpg onward per project, remain byte-identical to their sources, and sort naturally.
4. The sequence continues across batches, does not reuse deleted/uncertain values, skips occupied names, and blocks before namespace exhaustion.
5. Preview and Results remain aggregate-only and replace collision-renamed metrics with standard-format rename counts.
6. The unified private JSON manifest retains managed records after sorting and is atomically saved, validated, backed up, and bound to stable project ID.
7. Sort, Undo, class rename, and class deletion preserve or remove identity exactly as specified and never falsely report success after persistence failure.
8. Home-page Refresh runs in the background with a progress bar and Cancel button and applies no incomplete project's staged changes after cancellation or scan failure.
9. Refresh confirms expected paths without rehashing, reconciles one-to-one same-project moves by hash, preserves ambiguous records, and removes confirmed missing records.
10. Refresh never registers manually added JPEGs and never reconciles across projects.
11. Project cards and Review use managed-image totals and separately expose unregistered counts and locations.
12. Image Details displays all approved fields with the explicit encoded-byte label, selectable text, and working UUID/hash copy actions.
13. Active/backup corruption follows the approved recovery order, and Rebuild Tracking Records clearly discloses identity/provenance loss.
14. The implementation remains responsive and correct with representative projects containing 5,000 to 10,000 managed images.
15. All explicitly out-of-scope capabilities remain absent from this release.

18. Required Tests

18.1 Automated

- UUIDv4 generation, canonical serialization, uniqueness, duplicate-UUID rejection, and independence from equal hashes.
- Known SHA-256 fixtures, 1 MiB chunk boundaries, empty/final partial chunks, read failures, cancellation, and exact source/destination byte equality.
- Filename format, lexical ordering, per-project independence, batch continuation, gaps, case-insensitive collisions, and namespace exhaustion preflight.
- Source rejection inside the destination project and permitted Intake copying from another project into the destination.
- Unified manifest round trip, schema/project/path validation, malformed data rejection, atomic-save failure, locking, and backup promotion rules.
- Partial import persistence, temporary-file cleanup, no record for incomplete files, and restart recovery at every protected operation phase.
- Identity-preserving classification, Skip, repeated Undo, collision-driven sequence allocation, class rename path updates, and successful/failed class recycling.
- Refresh expected-path confirmation without hash calls; one-to-one same-project relocation; no cross-project match; unknown-file non-registration; and missing-record removal.
- Refresh ambiguity with equal hashes, invalid JPEG candidates, unsafe links, access failures, cancellation, and all-or-nothing per-project application.
- Managed/unregistered statistics, Review counts, project-card warning persistence, labeling denominator, and expandable path details.
- Active-manifest fallback to backup and rebuild behavior for standard/nonstandard names, recovered queue order, next sequence, and unknown provenance.
- Image Details read model, full values, encoded-byte label, clipboard actions, focus safety, long paths, and unrenderable-image behavior.
- Performance benchmarks with at least 10,000 records for manifest load/save, common sort updates, Refresh staging, and UI progress delivery.

18.2 Manual GUI Verification

- Import mixed .jpg/.jpeg batches and verify normalized names, aggregate Preview/Results, progress, cancellation, and unchanged sources.
- Verify Image Details at minimum and larger window sizes, common Windows display scaling, keyboard-only use, selection, copy buttons, and long values.
- Sort, Skip, Undo repeatedly, rename/delete classes, restart during controlled operations, and verify identity and recovery outcomes.
- Place registered images at new same-project paths, add unknown/invalid JPEGs, create ambiguous equal-hash candidates, delete tracked files, and verify Refresh summaries and details.
- Cancel Refresh at early, middle, and pre-commit stages and confirm no partial project reconciliation or missing-record removal.
- Exercise a project with 5,000 to 10,000 records and verify responsive repainting, useful progress, bounded memory behavior, and acceptable JSON update latency.

19. Risks, Assumptions, and Future Enhancements

JSON rewrite cost	A full per-project JSON rewrite may become noticeable as records grow. This release targets a single user and roughly 5,000 to 10,000 images; benchmark and optimize serialization without introducing SQLite.
Private identity	Because UUIDs are not embedded in JPEGs, external movement can be reconnected only through path and fingerprint evidence. Equal-content ambiguity must never be guessed.
External edits	A file edited in place can retain a stale stored hash. This is accepted; a future duplicate tool will rescan managed images before comparing them.
Manual files	Unknown JPEGs affect the unregistered metric but remain outside managed totals, sorting, and Review. Intake is the only supported registration path.
Crash windows	Filesystem and JSON replacement are not one transaction. Recovery records and deterministic ordering reduce, but do not eliminate, interruption complexity.
Sequence ceiling	The seven-digit namespace supports 9,999,999 project-lifetime allocations. Exhaustion blocks import; widening or changing the naming contract is a future release.
Future duplicate tool	May freshly hash every managed image, update stale fingerprints, group equal hashes within one project, and provide a separately approved review/removal workflow.

20. Reference Baselines

- prd/Completed_Releases/Outdoor_Vision_CV_App_Intake_Project_Tool_PRD_v1.0.pdf
- prd/Completed_Releases/Outdoor_Vision_CV_GUI_Home_Page_PRD_v1.0_Revised.pdf
- The current repository at implementation time.

APPROVAL EFFECT	After approval, this document becomes the implementation baseline for this enhancement. Any material change to identity semantics, hashing input, registration boundaries, filename format, or Refresh reconciliation requires explicit product approval.
-----------------	---